

PRODUCT AND CATEGORY MANAGEMENT SYSTEM

OBJECTIVES:

- Design separate tables for **Product** and **Category** to organize data efficiently.
- Define **primary key** and **foreign key** constraints to maintain data integrity.
- Store product information such as product name, category, price, and stock quantity.
- Perform basic database operations including **INSERT, UPDATE, DELETE, and SELECT**.
- Generate category-wise reports to display products under their respective categories.
- Improve inventory management by organizing products into different categories.

ENTITY DESIGN:

The system consists of two main entities:

1. Category Entity

The **Category** table stores information about different product categories. Each category is assigned a unique **CategoryID**, which acts as the primary key.

Attributes:

- **CategoryID** – Unique identifier for each category (Primary Key)
- **CategoryName** – Name of the product category

2. Product Entity

The **Product** table stores information about products available in the inventory. Each product belongs to one category.

Attributes:

- **ProductID** – Unique identifier for each product (Primary Key)
- **ProductName** – Name of the product
- **CategoryID** – Category to which the product belongs (Foreign Key)
- **Price** – Cost of the product
- **Stock** – Quantity of products available

PRIMARY AND FOREIGN KEY RELATIONSHIP:

Primary Key

A primary key uniquely identifies each record in a table.

- **CategoryID** is the primary key of the **Category** table.
- **ProductID** is the primary key of the **Product** table.

These keys ensure that duplicate records are not allowed.

Foreign Key

The **CategoryID** in the **Product** table is a foreign key that references the **CategoryID** in the **Category** table.

This relationship:

- Connects products with their respective categories.
- Prevents invalid category entries.
- Maintains referential integrity between both tables.

Relationship:

- One Category → Many Products
- One Product → One Category

SQL IMPLEMENTATION:

The SQL implementation is divided into the following steps:

Step 1: Create Tables

The **Category** and **Product** tables are created with appropriate data types and constraints. Primary keys uniquely identify records, while the foreign key establishes the relationship between the tables.

Step 2: Insert Records

Category details such as **Electronics, Clothing, and Books** are inserted into the Category table. Product information including product name, category, price, and stock is inserted into the Product table.

Step 3: Update Records

The SQL **UPDATE** statement is used to modify existing product information. In this project, the price and stock of the Laptop are updated.

Step 4: Delete Records

The **DELETE** statement removes unwanted records from the Product table. Here, the **Database Book** product is deleted.

Step 5: Generate Reports

A **JOIN** operation is used to combine the Product and Category tables and display category-wise product details.

OPERATIONS PERFORMED:

1. Create Operation

Created the **Category** and **Product** tables with necessary fields and constraints.

2. Insert Operation

Inserted sample records into both tables to store category and product information.

3. Update Operation

Updated the Laptop price from ₹55,000 to ₹60,000 and changed the stock quantity from 10 to 8.

4. Delete Operation

Deleted the **Database Book** record from the Product table.

5. Select Operation

Retrieved product details category-wise using an SQL **JOIN** query.

CATEGORY-WISE REPORTS GENERATED:

The category-wise report displays products along with their categories. The report includes:

- Category Name
- Product Name
- Product Price
- Available Stock

The report is generated using an **INNER JOIN** between the **Category** and **Product** tables based on the **CategoryID** field.

Sample Output

Category Name	Product Name	Price	Stock
Clothing	T-Shirt	800.00	50

Category Name	Product Name	Price	Stock
Electronics	Laptop	60000.00	8
Electronics	Smartphone	25000.00	15

CONCLUSION:

The **Product and Category Management System** is a simple SQL-based database application that efficiently manages product information. It demonstrates the use of **primary keys**, **foreign keys**, and **CRUD operations** to maintain organized and accurate data. The category-wise report helps users easily view products under each category, making inventory management more effective. This project provides a strong foundation for developing larger inventory and retail management systems.